

ASP.NET Core Controller Based Web API with EF Core Code First Approach

Agenda:

- Creating ASP.NET Core Controller Based Web API Project
- Install EF Core Packages
- SQL Server Connection Connection String
- Create Model (Entity Classes)
- DbContext (Datalayer) with DbSet(s)
- Register DbContext (Dependency Injection)
- Apply Migration & Update Database
- Perform CRUD (GET, GET by ID, POST, PUT, DELETE)

From Insurance Domain we will be using Policy section only.

Policy Model

Column	Type
PolicyId	int
PolicyNumber	string
CustomerName	string
PolicyType	string
PremiumAmount	decimal
StartDate	DateTime
IsActive	bool

```
public class Policy
{
    public int PolicyId { get; set; }
    public string PolicyNumber { get; set; }
    public string CustomerName { get; set; }
    public string PolicyType { get; set; }
    public decimal PremiumAmount { get; set; }
    public DateTime StartDate { get; set; }
    public bool IsActive { get; set; }
}
```

Sample Records

Id	Policy Number	Customer	Type	Premium
1	POL1001	John	Health	25000
2	POL1002	David	Motor	18000

Install EF Core Packages

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Or use NuGet Package Manager (Visual Studio)

Create Models and Data Folders

Add Policy class as stated above.

Add InsuranceDbContext class in Data Folder

```
using InsurancePolicyApi.Models;
using Microsoft.EntityFrameworkCore;

namespace InsurancePolicyApi.Data
```

```
{
    public class InsuranceDbContext : DbContext
    {
        public InsuranceDbContext(DbContextOptions<InsuranceDbContext>
options): base(options){}

        public DbSet<Policy> Policies => Set<Policy>();
    }
}
```

Go to Appsettings.json and add connection string to connect to SQL Server:

```
"ConnectionStrings": {
    "csInsDB": "Data Source=(localdb)\\MSSQLLocalDB;Initial
Catalog=HFInsDB; Integrated Security=true; TrustServerCertificate=true"
},
```

Go to Program.cs and register dbcontext

```
//Configure the ConnectionString and DbContext class
builder.Services.AddDbContext<InsuranceDbContext>(options =>
{
    options.UseSqlServer(builder.Configuration.GetConnectionString("csInsDB"));
});
```

Now need to get ready with Database → Fire following commands or use Package Manager Console in case of Microsoft Visual Studio.

...> dotnet ef migrations add Initial or in PM Console: PM> Add-Migration Initial

...> dotnet ef database update or in PM Console: PM> update-database

Go to Controllers folder and add New Controller named PoliciesController and add action methods for GET, GET by ID, POST, PUT and DELETE.

```
using InsurancePolicyApi.Data;
using InsurancePolicyApi.Models;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace InsurancePolicyApi.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class PoliciesController : ControllerBase
    {
        private readonly InsuranceDbContext _context;

        public PoliciesController(InsuranceDbContext context)
        {
            _context = context;
        }

        // GET: api/Policies
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Policy>>> GetPolicies()
```

```

    {
        return await _context.Policies.ToListAsync();
    }

    // GET: api/Policies/4
    [HttpGet("{id}")]
    public async Task<ActionResult<Policy>> GetPolicy(int id)
    {
        var policy = await _context.Policies.FindAsync(id);

        if (policy == null)
        {
            return NotFound();
        }

        return policy;
    }

    // POST: api/Policies
    [HttpPost]
    public async Task<ActionResult<Policy>> PostPolicy(Policy policy)
    {
        _context.Policies.Add(policy);
        await _context.SaveChangesAsync();

        return CreatedAtAction("GetPolicy", new { id = policy.PolicyId },
policy);
    }

    // PUT: api/Policies/4
    [HttpPut("{id}")]
    public async Task<IAActionResult> PutPolicy(int id, Policy policy)
    {
        if (id != policy.PolicyId)
        {
            return BadRequest();
        }

        _context.Entry(policy).State = EntityState.Modified;

        try
        {
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!PolicyExists(id))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }

        return NoContent();
    }

    // DELETE: api/Policies/4
    [HttpDelete("{id}")]

```

Free, Open-Source, Cross-Platform Framework

```
public async Task<IActionResult> DeletePolicy(int id)
{
    var policy = await _context.Policies.FindAsync(id);
    if (policy == null)
    {
        return NotFound();
    }

    _context.Policies.Remove(policy);
    await _context.SaveChangesAsync();

    return NoContent();
}

private bool PolicyExists(int id)
{
    return _context.Policies.Any(e => e.PolicyId == id);
}
}
```

☪*☪ Routing: Direct your hard work to the right goals ☪*☪